

Computer Programming Summer Interest Group

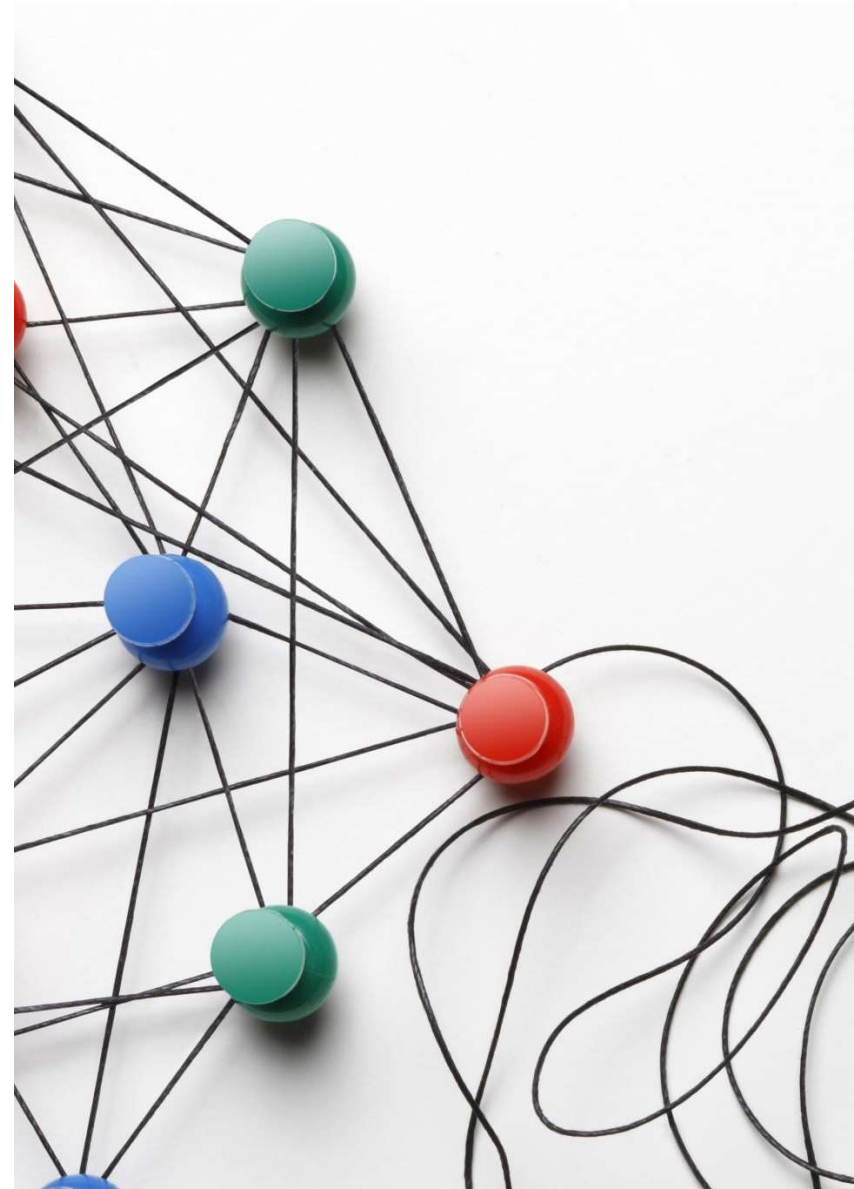
Session 20 / Concurrency 3

John 14:15-18

“If you love me, you will keep my commandments. And I will ask the Father, and he will give you another Helper, to be with you forever, even the Spirit of truth, whom the world cannot receive, because it neither sees him nor knows him. You know him, for he dwells with you and will be in you. I will not leave you as orphans; I will come to you.”

Objectives

- By the end of this session, you will be able to:
 - Explain how multiprocessing differs from threading.
 - Use the `multiprocessing.Process`` class to run functions concurrently.
 - Communicate between processes using `Queue`` and `Pipe``.
 - Apply synchronization tools like `Lock`` and `Manager``.





Python multiprocessing package

- The Python Global Interpreter Lock (GIL) ensures tasks are run on a single thread
 - Limits asyncio to running in a single thread
 - Python threads are still implemented to use the GIL, so they do not achieve true multi-threading, though there is a move to do so
- The multiprocessing package overcomes this limitation of the GIL and allows workloads to be distributed across compute cores

Python multiprocessing Package Useful Components

Component	Description
<code>multiprocessing.Process(target, args=())</code>	Creates and runs a new process.
<code>multiprocessing.Pool(processes)</code>	Creates a pool of worker processes.
<code>multiprocessing.Queue()</code>	FIFO queue for inter-process communication.
<code>multiprocessing.Pipe()</code>	Returns a pair of connection objects for communication.
<code>multiprocessing.Lock()</code>	Prevents concurrent access to shared resources.
<code>multiprocessing.Manager()</code>	Provides shared objects like lists and dicts.

Inter-Process Communication

- Processes do not share memory
- Communication between them uses “Inter-Process Communication” (IPC)
- IPC is a service the operating system provides to pass messages between different processes running on the host.
- The Queue, Pipe and Manager components of the multiprocessing package use IPC to pass data between processes
- The data must be “serializable”, capable of being converted into a byte stream
 - Several Python objects are not serializable:
 - File handles, network sockets and lambda functions

Simple case, running multiple processes

- We create new processes using the Process class
- The Process constructor wraps a function and its arguments
- The function will be run with its arguments in a separate process
- Use the .start method to start the process
- Use the .join method to wait for the joined process to complete... similar to the way the join method works for threads.
- You can use os.getpid() to get the process identifier (PID) of the process that calls it.

```
import time
from multiprocessing import Process

def wait_task(task_id: int, delay: float):
    print(f"Task {task_id} started - {delay=} ")
    time.sleep(delay)
    print(f"Task {task_id} finished")

def main():
    processes = [
        Process(target=wait_task, args=(i, i + 1))
        for i in range(3)
    ]

    for process in processes:
        process.start()

    for process in processes:
        process.join()

if __name__ == "__main__":
    main()
```

Simple case, running multiprocessing pool

- In the previous example we needed to manage each process manually
- Without adding constraints, this approach may lead to run-away resource usage
- Use of a process pool limits the number of processes which can be added
- The example to the right starts 6 processes but has a thread pool only of size 3
- The first three processes start more or less concurrently
- The remaining processes will start as each of the initial processes concludes
- The Pool is responsible for starting each process and starting pending processes as the original finish
- We pass the desired time delay along with the actual delay for each call to `time.sleep`
- See [Multiprocessing Pool apply\(\) vs map\(\) vs imap\(\) vs starmap\(\)](#) for alternatives to `starmap`.

```
import time
import os
from multiprocessing import Pool

def wait_task(task_id: int,
              delay: float) → tuple[float, float]:
    print(f"starting {task_id=} {delay=}")
    start = time.time()
    time.sleep(delay)
    return delay, time.time() - start

def main():
    pool = Pool(processes=3)
    results = pool.starmap(
        wait_task,
        [(i, i + 1) for i in range(6)])
    pool.close()
    pool.join()
    print(results)

if __name__ == "__main__":
    main()
```

Use of a Queue to pass data

- A Queue object allows data to be passed from one process to another.
- A Queue only allows data to flow in one direction
- Data is relayed using the Operating System implementation of Interprocess Communication (IPC)
- Items are sent from the sender by using the 'put' method.
- Items are received at the recipient process by using the 'get' method
- The empty method is used to check if the queue is empty.

```
from multiprocessing import Process, Queue

def producer(q: Queue):
    for i in range(5):
        q.put(i)
        print("Produced", i)

def consumer(q: Queue):
    while not q.empty():
        item = q.get()
        print("Consumed", item)

def main():
    q = Queue()
    p1 = Process(target=producer,
                 args=(q,))
    p2 = Process(target=consumer,
                 args=(q,))

    p1.start()
    p2.start()
    p1.join()
    p2.join()

if __name__ == "__main__":
    main()
```

Use of a Pipe to pass data bidirectionally

- A Pipe allows processes to communicate bidirectionally.
- Pipe() returns a tuple with two “Connection” objects which are connected to each other
 - Note “conn1, conn2 = Pipe()”
- Either connection can send data to or receive data from the other end of the connection
- The “conn.poll” method waits until there is data to receive over the connection
 - It will return False if not data is received within the timeout period
 - It will return True if data is received within the timeout period
- The “conn.recv” receives the data sent from the other end of the pipe.

```
import time
import os
from multiprocessing import Process, Pipe
from multiprocessing.connection import Connection

def pong(conn: Connection, name: str, counter: int = 5):
    while counter > 0:
        conn.send(f"Hello from {name}")
        time.sleep(0.1)
        if conn.poll(timeout=1.0):
            reply = conn.recv()
            print(f"{name} received: '{reply}'")
            counter -= 1
        else:
            break

def main():
    conn1, conn2 = Pipe()
    p1 = Process(target=pong, args=(conn1, "P1"))
    p2 = Process(target=pong, args=(conn2, "P2"))
    p1.start()
    p2.start()
    p1.join()
    p2.join()

if __name__ == "__main__":
    main()
```

Use a Lock to prevent race conditions

- A race condition occurs when two processes attempt to alter the same resource at the same time
- To prevent this Python provides an inter-process locking mechanism
- The state of the lock is passed between processes using inter-process communication (IPC)
- When one process holds the lock, no other process may take action which requires it to hold the lock
- Waiting on the lock concludes when the process holding the lock releases it.
- The “with lock” holds the lock for the steps in the code block in the “with” context manager.
- The lock is released when control exits the “with” context manager.

```
import time
import os
import random
from multiprocessing import Process, Lock

def worker(lock: Lock, id: int):
    for i in range(3):
        print(f"Process {id} waiting for lock")
        with lock:
            print(f"Process {id} has the lock (iteration {i})")
            time.sleep(random.uniform(0.1, 0.5))
            print(f"Process {id} is releasing the lock")
            time.sleep(random.uniform(0.1, 0.2))

def main():
    print(f"Main process started.")
    lock = Lock()
    processes = [Process(target=worker, args=(lock, i))
                  for i in range(3)]

    for p in processes:
        p.start()

    for p in processes:
        p.join()

    print(f"Main process finished.")

if __name__ == "__main__":
    main()
```

Use a Manager to share a resource

We can use a “Manager” to share a resource across different processes

We create a Manager object and then create a shared list or dict which can be passed to each one or more Process instances

If making changes, a lock should be used to prevent race conditions

The Manager instance should be created within a context manager to ensure it is properly cleaned up when leaving the scope of the context manager.

```
import time
import random
from multiprocessing import Process, Manager

def worker(lock, id: int, shared_dict: dict):
    for i in range(3):
        print(f"Process {id} waiting for lock...")
        with lock:
            print(f"Process {id} has the lock.")
            shared_dict[f"process_{id}"] = (i, random.randint(1, 100))
            print(f"Process {id} updated dict: {shared_dict}")
            time.sleep(0.1) # Simulate work
        print(f"Process {id} released the lock.")
        time.sleep(random.uniform(0.1, 0.3))

def main():
    with Manager() as manager:
        shared_dict = manager.dict()
        lock = manager.Lock()
        print(f"Initial dictionary: {shared_dict}")

        processes = [
            Process(target=worker, args=(lock, i, shared_dict))
            for i in range(3)]

        for p in processes:
            p.start()

        for p in processes:
            p.join()

        print(f"Final dictionary: {shared_dict}")

if __name__ == "__main__":
    main()
```



Activities

- I have written a program the generates a Mandelbrot image in a single process. See `mandel.py` in the github repo folder for this session.
- Modify it to use a multiprocessing pool
 - Hint: You should only need to modify the “`generate_data`” function to compute each line in a separate process... don’t try to compute each point in a separate process.
- The pool size should not exceed the number of processing cores you have on your computer.
- You can read more about the [Mandelbrot set](#)